## PART A: STRUCTURED QUESTION [70 MARKS]

*Part A consists of five (5) structured questions. Write your answers in the answer booklet.*

**Question 1 (10 marks)**

Analyze the following Java program in Figure 1.1 below :

```
1  import java.util.ArrayList;
2  import java.util.Vector;
3
4  public class AQList{
5      public static void main(String[] args) {
6          ArrayList<String> arrayList = new ArrayList<>();
7          arrayList.add("Banana");
8          arrayList.add("Olive");
9          arrayList.add("Date");
10         arrayList.add("Fig");
11         System.out.println("List: " + arrayList);
12
13         Vector<String> vector = new Vector<>();
14         vector.add("Ants");
15         vector.add("Bee");
16         vector.add("Cow");
17         vector.add("Dog");
18         System.out.println("Vector: " + vector);
19
20         arrayList.set(1, "Grape");
21         arrayList.remove(0);
22
23         vector.remove("Cow");
24         vector.setElementAt("Fish",2);
25
26         System.out.println("List: " + arrayList);
27         System.out.println("Vector: " + vector);
28 }
29 }
```

Figure 1.1: `AQList` class definitions

Answer the given question based on Figure 1.1 `AQList` class

i) The output for line 11 is : (2 marks)

ii) The output for line 18 is : (2 marks)

iii) The output for line 26 is : (3 marks)

iv) The output for line 27 is : (3 marks)

**Question 2 (15 marks)**

Analyze the class diagram and incomplete program that represents the class `House`, `Furniture`, and `Room` relationship.

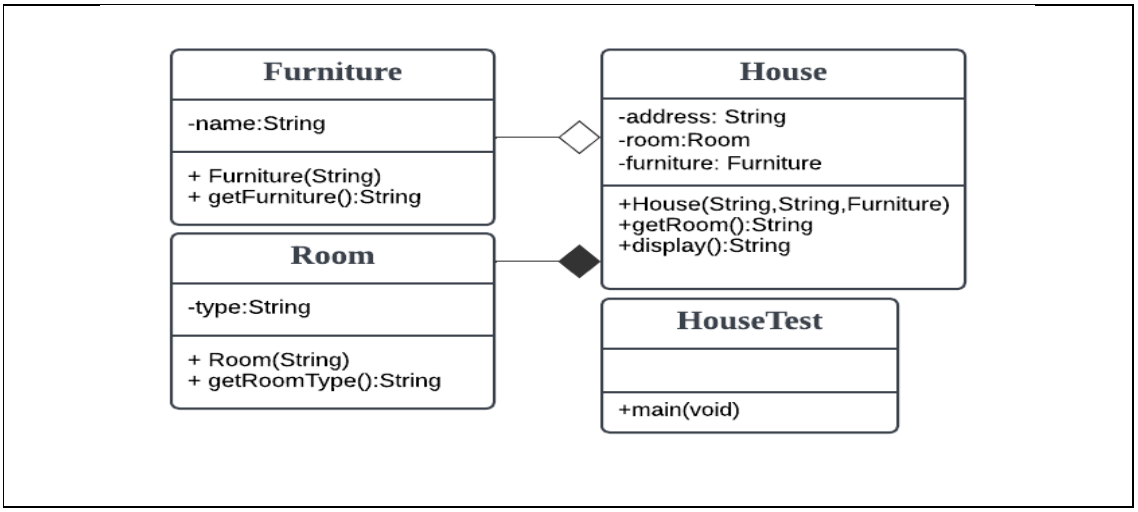


Figure 2.1: Class Diagram of **`House`**, **`Furniture`**, **`Room`**

Given Figure 2.2, class `HouseTest` for the main method implementation.

```
// class HouseTest
public class HouseTest {
    public static void main(String[] args) {
        Furniture fur1 = new Furniture("Sofa");
        House house = new House("No 2 Taman U", "Bedroom",fur1);
        System.out.println(house.display());
    }
}
```

Figure 2.2: **`HouseTest`** class definitions

```
//class House
class House {
    ____________________
    ____________________
    ____________________

    public House(___________, __________, __________) {
        ____________________
        ____________________
        ____________________
    }

    public String getRoom(){
            return ________________;
        }

        public String display(){
            return ________________________________;
    }
}
```

Figure 2.3: **`House`** class definitions

```
1   //class Room
2   class Room {
3           private String type;
4
5           public Room(String type) {
6               this.type = type;
7           }
8
9           public String getRoomType() {
10              ____________
11          }
12      }
```

Figure 2.4: `Room` class definitions

```
1   // class Furniture
2   class Furniture {
3       private String name;
4
5       public Furniture(String name) {
6           this.name = name;
7       }
8
9       public String getFurniture() {
10          ____________
11      }
12  }
```

Figure 2.5: `Furniture` class definitions

```
Address:  No 2 Taman U
Furniture:  Sofa
Room:  Bedroom
```

Figure 2.6: Output

Construct the correct codes according to the given class diagram (Figure 2.1). The code should generate the specified output as in Figure 2.6.

i) Figure 2.3 at line 3-line 5: define the attributes of the `House` class (3 marks)

ii) Figure 2.3 at line 7-line 10: complete code of `House` class constructor. (6 marks)

iii) Figure 2.3 at line 14: complete code of `getRoom()` method that return a value of room type (1 mark)

iv) Figure 2.3 at line 18: complete code of `display()` method that displays the output as shown in Figure 2.6. (3 marks)

v) Figure 2.4 at line 10: complete code of `getRoomType()` (1 mark)

vi) Figure 2.5 at line 10, complete code of `getFurniture()`. (1 mark)

**Question 3 (15 marks)**

3.1 Examine the following Java source code in Figure 3.1.

```
class Item {
    private String name;

    public Item(String name) {
    // i.assign Item class attribute's values
        ________________
    }

    public String getName(){
        return name;
    }
}

class Product extends Item {
    private double price;
    private int quantity;

    public Product(String name, double price, int quantity) {
    // ii) assign subclass and superclass attribute values
       _________________
       _________________
       _________________
    }

    public Product(String name, double price) {
      iii) assign subclass and superclass attribute values
       ________________
       ________________

    }

  public void printInfo() {
     iv) Print name, quantity, and price
       ___________________________________
       ___________________________________
       ___________________________________
}

public class TestProduct {
    public static void main(String[] args) {
        Product product1 = new Product("Book", 15.99, 10);
        Product product2 = new Product("Pen", 1.49);
        product1.printInfo();
        product2.printInfo();
 }
}
```

Figure 3.1: **Product, Item**, and **TestProduct** class definitions

Answer the following questions based on the classes defined in Figure 3.1 above.

i) Assign values for `Item` superclass attributes (1 mark)

ii) Assign values to subclass attributes and its superclass inherited attribute (3 mark)

iii) Assign values to subclass attributes and its superclass inherited attribute (2 marks)

iv) Print the `name`, `price`, and `quantity` (3 marks)

3.2 Analyze the following Java source code in Figure 3.2.

```
class Perfume {

    public Perfume() {
        System.out.println("Super-constructor");
    }

    public void scent() {
        System.out.println("Pleasant scent.");
    }
}

class CarPerfume extends Perfume {

    public CarPerfume() {
        System.out.println("Sub-constructor");
    }

    public void scent() {
        System.out.println("Refreshing scent.");
    }
}

public class TestPerfume {
    public static void main(String[] args) {
        CarPerfume myCarPerfume = new CarPerfume();
        myCarPerfume.scent();
    }
}
```

Figure 3.2: `Perfume`, `CarPerfume`, and `TestPerfume` class definitions

Based on the Java source code in Figure 3.2 above, complete the following tasks (i – iii):

(i) Draw the UML class diagram that represents of Java source code in Figure 3.2. (2 marks)

(ii) What is the output of the Java source code in Figure 3.2? (3 marks)

(iii) What is the significance of the `scent()` method in both the Perfume and `CarPerfume` classes. (1 mark)

**Question 4 (15 Marks)**

4.1 Examine the program in Figure 4.1. Write the correct output of this program. (4 marks)

```
class One {
 protected int x = 4;

void setX(int a){
  x=a;
 }
protected int getX(){
 return x;}
}

class Two extends One {
 protected int x = 8;

 protected int getX(){
 return x; }

public int getB(){
 return x;
}
}

public class Test{
 public static void main(String [] args){
  One a = new One();
  One b = new Two();
  System.out.println(a.getX());
  System.out.println(b.getX());
  System.out.println(a.x);
  System.out.println(b.x);
 }
}
```

Figure 4.1: `One`, `Two`, and `Test` class definitions

4.2 Examine the following Java program that demonstrates the interface and abstract class through the definition of **`Salary`**, **`Employee`**, **`Manager`**, **`Programmer`** and **`TestSalary`** classes. Fill in the blanks with the correct code to complete the program. The expected output shown in figure 4.2. (11 marks)

```
interface Salary {
    double calculateSalary();
}
abstract class Employee implements Salary {
   protected String name;
   _________ String role;

    public Employee(String name, String role) {
        this.name = name;
        this.role = role;
    }

    public ________ double calculateSalary();

    public void printDetails() {
        System.out.println("Role: " + role);
        System.out.println("Name: " + name);
        System.out.println("Salary: RM " + ________);
    }
}
class Manager extends _________{
    public Manager(String name) {
        ______(name, "Manager");
    }

    public double _________ {
        return 9000.0;
    }
}
class Programmer extends Employee {
    public Programmer(String name) {
        ______ (name, "Programmer");
    }

    public double _________{
        return 5600.0;
    }
}
public class TestSalary {
    public static void main(String[] args) {
        Employee manager = new ________ ("Faiz Ali");
        Employee programmer = new _________ ("Ely Tan");
        ________.printDetails();
        System.out.println();
        ________.printDetails();
    }
}
```

```
Manager: Faiz Ali
Role: Manager
Salary: RM 9000.0

Programmer: Ely Tan
Role: Programmer
Salary: RM 5600.0
```

Figure 4.2: Expected output

**Question 5 (15 Marks)**

5.1 Given the class `CalculatorDemo` in Figure 5.1 below:

```
 1  public class CalculatorDemo {
 2      public static void main(String[] args) {
 3          boolean flag = true;
 4          if (flag = false) { //
 5              System.out.println("This is a syntax error.");
 6          }
 7
 8          int i = 0;
 9          int j = 10;
10          int x = calculate(i,j);
11          int y = j/i;
12          int[] arr = new int[5];
13          System.out.println(arr[10]);
14
15      }
16
17      public int calculate (int a, int b){
18          int result;
19          result = b + a;
20
21    }
22   }
```

Figure 5.1: Java program of class `CalculatorDemo`

Trace the provided Java program and identify the line number and type of error (syntax error, runtime error, or logical error) at each problematic point in the program.

| Line Number | Type of error |
|---|---|
| | |
| | |
| | |
| | |

(8 marks)

5.2 Given the class `ExceptionExample` in Figure 5.2 demonstrating exception handling for array creation, input validation, and arithmetic operations using try-catch blocks. Complete the following Java program by filling in the blanks: (7 marks)

```
import java.util.Scanner;

public class ExceptionExample {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

   try {
    // Part 1: Creating an array with user-defined size
    System.out.println("Enter the size of the array:");
    int size = scanner.nextInt();
    if (size <= 0) {
        throw new ____________("Array size must be a positive
        integer."); // 1.5 mark
    }

    int[] array = new int[size];

    for (int i = 0; i <= size; i++) {
        System.out.println("Enter element " + i + ":");
        array[i] = scanner.nextInt();
    }

    // Part 2: Performing division
    System.out.println("Enter a divisor:");
    int divisor = scanner.nextInt();
    if (divisor == 0) {
        throw new ________("Cannot divide by zero."); // 1.5 mark
    }
    int result = element / divisor;
    System.out.println("Result of division: " + result);

    // Part 3:
    } catch (______ e)1.5 mark {
        System.out.println("Index out of bounds. Please enter a
        valid index.");
    } catch (______ e)1.5 mark {
        System.out.println("Arithmetic error: " +
        e.________()) 1 mark;
    } finally {
            scanner.close();
    }
  }
}
```

Figure 5.2: `ExceptionExample` program.

## PART B: PROGRAMMING QUESTION [30 MARKS]

*Part B consists of **ONE (1)** programming question. Write your answers in the answer booklet.*

Write a complete Java program that represents the UML class diagram in Figure 1 and produces the expected output depicted in Figure 2. Implement all the classes (excluding the **Order** and **FoodOrderingSystem** classes) with the specified instance variables (attributes) and methods as outlined in the diagram. The purpose of each method is as the name implies, and some of them are further explained below. Note that the definitions of the **Order** and **FoodOrderingSystem** classes are fully given in Figure 3.

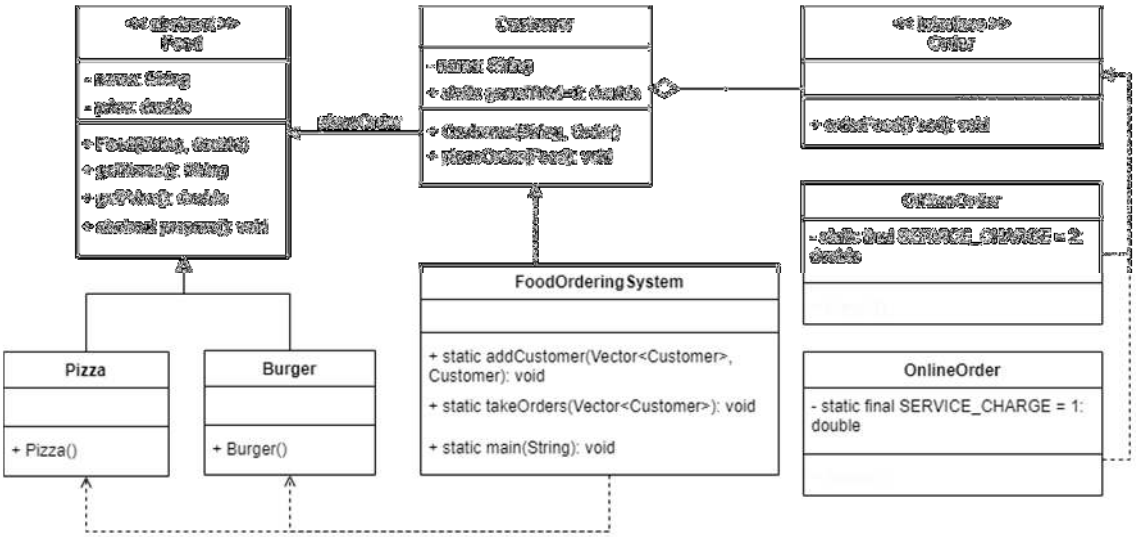


Figure 1: UML Class Diagram

```
Preparing Hawaiian Chicken...
Hawaiian Chicken ordered online.
Total: RM41.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered online.
Total: RM19.35

Preparing Hawaiian Chicken...
Hawaiian Chicken ordered offline.
Total: RM42.90

Preparing Double Cheeseburger...
Double Cheeseburger ordered offline.
Total: RM20.35

Grand Total Sales = RM124.50
```

Figure 2: Expected output of the program

You are required to do the following tasks:

a) Define a class named **OnlineOrder**: (5 marks)

- This class should implement the **Order** interface.
- Declare a constant (SERVICE_CHARGE) representing the service charge for online orders.
- Define an `orderFood()` method that prepares the food, calculates the cost for online order food by adding the price of food with the service charge, which is RM1.00 for online orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).

b) Define a class named **OfflineOrder**: (5 marks)

- This class should implement the **Order** interface.
- Declare a constant (SERVICE_CHARGE) representing the service charge for offline orders.
- Define an `orderFood()` method that prepares the food, calculates the cost for offline order food by adding the price of food with the service charge, which is RM2.00 for offline orders and updates the grand total sales. Then, display the order and the cost of the order on the screen (refer to Figure 2).

c) Define an abstract class named **Food**: (6 marks)

- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
- Define `getName()` method that returns the food name.
- Define `getPrice()` method that returns the food price.
- Declare an abstract method named `prepare()`. Subclasses must define this method.

d) Define a class named **Pizza**: (4 marks)

- This class should inherit from the **Food** class.
- Define a constructor and initializes the superclass's attributes with "Hawaiian Chicken" as name and RM40.90 as price.
- Define `prepare()` method that displays pizza preparing message.

e) Define a class named **Burger**: (4 marks)

- This class should inherit from the **Food** class.
- Define a constructor and initializes the superclass's attributes with "Double Cheeseburger" as name and RM18.35 as price.
- Define `prepare()` method that displays burger preparing message.

f) Define a class named **Customer**: (6 marks)

- Define a constructor that takes arguments and initializes the member attributes with passed arguments.
- Define `placeOrder()` method that places the food order made by the customer.

```java
// FoodOrderingSystem.java
import java.util.*;

interface Order {
  void orderFood(Food food);
}

class FoodOrderingSystem {
  //Method to add a customer to the system
  public static void addCustomer(Vector<Customer> cList, Customer c) {
    cList.add(c);
  }

  // Method to take orders from all customers in the system
  public static void takeOrders(Vector<Customer> cList) {
    for (Customer c: cList) {
      c.placeOrder(new Pizza());
      c.placeOrder(new Burger());
    }
  }

  public static void main(String[] args) {
    // Create a new customer list
    Vector<Customer> cList = new Vector<>();

    //Add customers to the system
    addCustomer(cList, new Customer("Haris", new OnlineOrder()));
    addCustomer(cList, new Customer("Aliya", new OfflineOrder()));

    //Take orders from all customers
    takeOrders(cList);
    System.out.printf("Grand Total Sales = RM%.2f\n",
    Customer.grandTotal);
  }
}
```

**Figure 3: Order** and **FoodOrderingSystem** class definitions